

CANDY

DIALOGUE ENGINE

Condition Commands

1.1.2

Condition commands are used when you need dialogue to run specific lines or branch in different directions depending on conditions.

Conditions

`§If` / `§Elif` / `§Else` Chains

The `§If`, `§Elif` and `§Else` commands can be used very similarly to `if` / `elif` / `else` in GDScript.

Add an `§If` command to start a chain, indicate the condition that must be fulfilled and what should happen when it is, and that's it.

Then, add `§Elif` commands or an `§Else` command in immediate succession, as needed.

Keep in mind that `§Elif` and `§Else` will only work if the chain is not broken prior: `§If` starts a chain, and the chain will break when it encounters any spoken line or any command other than `§Elif`, `§LM` or `§Comment`.

Just like in GDScript, `§Else` will break a chain, while adding another `§If` command will start a new one.

`§LM` and `§Comment` are exempted from breaking chains because they are essentially just information inserted in the dialogue script, not actual commands that have any effect on their own.

Note that placing an `§LM` command in the middle of a condition chain serves no purpose: if you transition to that `§LM` directly, the `§If` that started the chain won't be processed. Therefore, any `§Elif` or `§Else` after the `§LM` will fail because no chain will have been started. See the Transitions guide for details about the `§LM` command and transitions.

Some examples:

<code>§If</code>	→ Start new chain 1
<code>§Elif</code>	→ Continue chain 1
<code>§Else</code>	→ End chain 1

<code>§If</code>	→ Start new chain 1
<code>§If</code>	→ End chain 1, start new chain 2
<code>§Elif</code>	→ Continue chain 2

<code>\$If</code>	→ Start new chain 1
<code>\$Elif</code>	→ Continue chain 1
<code>\$Call</code>	→ End chain 1
<code>\$Elif</code>	→ No effect: chain 1 already ended

<code>\$If</code>	→ Start new chain 1
<code>\$Else</code>	→ End chain 1
<code>\$Elif</code>	→ No effect: chain 1 already ended

\$For & \$While Loops

Candy DE also provides `$For` and `$While` commands to emulate GDScript for and while loops.

They work similarly to `$If`, `$Elif` and `$Else`: write the condition that must be fulfilled, and indicate what happens when it is.

The `$While` command has a limit to the number of loops it can perform! This is an optional safety feature to avoid accidental infinite looping. By default, it is set at 1000 loops.

To change the loop limit or disable it altogether, open [Candy_Properties.gd](#) and look for `while_safety_break`. Set it to a negative value to disable it.

```
var while_safety_break = 1000
```

Condition Writing

Candy DE uses Godot's native `Expression.execute()` method to parse comparison strings.

To write conditions, just write them like you would in GDScript, with one small distinction: don't write 'if', 'elif', 'else', 'for' or 'while' -- the engine knows what type of condition you're using by looking at the command.

You'll also need to use the variable symbols (default '£', '\$' and '€') to designate variables (see the [Variables](#) guide for details).

As an example:

£globals.player_health < 10

You can compare variables to one another, use 'AND/and/&&', 'OR/or/||' and 'not/!' operators, and even use parentheses:

£globals.var_1 == £globals.var_2 OR (£globals.var_1 > 3 AND globals.var_2 == true)

'NOT' (uppercase) is not supported, and neither is 'XOR/xor/^^'

Note that the Expression class may have limitations over regular GDScript. Also, at this time, the official Godot documentation doesn't seem to mention condition operators (we may have missed something); however, the Expression class is confirmed working for comparisons by our own testing.

Note also that you can't chain comparisons inside an expression. For example, this isn't possible:

2 < £globals.var_1 < 8

This will fail because Expression.execute() will first convert '2 < £globals.var_1' to 'true'; this then results in comparing 'true < 8' which isn't possible.

Instead, write:

£globals.var_1 > 2 AND £globals.var_1 < 8

In the case of the \$For command, you can use the operator 'in', combined with an **iterator** of your choice. For example:

i in £globals.my_array

i in range(3)

i in range(1, 5)

i in range(1, 6, 2) → '2' is an increment amount

You'll probably want to retrieve the value of the iterator. It automatically gets stored to the **for_** dictionary in **Candy_Engine.gd**. To retrieve the value, for example through a \$Set command that runs at each iteration, you would write:

£self.for_["i"]

To clarify, the iterator can have any name you want, not just "i" (e.g. "node", "object", "child", etc.). Just like in GDScript, if you have nested \$For commands, make sure to use different operators. GDScript displays an error if you don't, but Candy DE will quietly overwrite the iterator values.

Super Variables

Instead of providing a condition string directly to a condition command, you can store an entire condition string in a variable, then reference that variable in the command data. For example:

var condition_1 = "£globals.player_hp >= 100"

However, since the engine expects to find variables inside the condition string by default, the base variable symbols (default: £, \$ and €) will not work for replacing the entire string with a variable reference.

For example, if you just write "£globals.condition_1" in the condition string, the engine will read this as an incomplete condition: it expects a value to compare the variable to, and an operator to tell it what kind of comparison to make.

Instead, you must use the 'super' variable symbols (default: '££', '\$\$' and '€€'), like this:

££globals.condition_1

This will make the engine replace the entire condition string with the value of condition_1 above:

£globals.player_hp >= 100

For variables in external files (normally stored or read with 'v_res://' or 'v_user://'), use 'sv_res://' or 'sv_user://'.

Consequences

To define what happens when a condition is fulfilled, provide a command name in the Condition Effect field.

In Candy Dialogue Creator, this will display the data fields for the provided command. Just fill data in these fields as normal to configure the command.

Only one command can be added in this way to a condition command, and those commands can't be other condition commands.

To run multiple commands when a condition is fulfilled, or to nest multiple condition commands inside each other, you must use a transition command: either §Jump or §Bridge, and simply transition to a Conversation/Block/Line that features all the commands you want to run.

Potential Errors

Setting up conditions isn't difficult, in fact it's very intuitive. But just like when coding, you'll need to be careful what you actually make your conditions do. The most likely errors would be incorrectly chaining §If, §Elif and §Else commands, or using the wrong transition type.

If you regularly test your dialogues while you work, you'll immediately notice such errors and you'll be able to easily correct them.